

Arithmetic, Ratios, and Log Returns

A single-column field-guide manual for quant research foundations

What this manual trains

Turn raw prices into usable return series. Understand percent change, ratios, simple returns, log returns, compounding, cumulative returns, annualization, and the implementation traps that corrupt backtests. The focus is fast recall plus practical system design.

simple: $r_t = P_t / P_{t-1} - 1$

gross: $G_t = 1 + r_t = P_t / P_{t-1}$

log: $l_t = \ln(P_t / P_{t-1})$

How to Use This Manual

PURPOSE

Use this as a fast recall system, not as a linear textbook. Each concept page is a standalone reference card. Read the top summary first, then scan formula blocks, examples, implementation notes, and failure modes.

READING SEQUENCE

- First pass: read Meaning, Memory jogger, and Quant use only.
- Second pass: work the calculation and formulas.
- Third pass: connect the page to your data pipeline, backtest engine, and feature store.
- Later review: use the recall questions and failure-mode pages to test whether the concept is actually usable.

MANUAL STRUCTURE

Module	What to look for	Why it matters
Meaning	Plain-English definition	Prevents fake understanding of notation
Formula block	Exact computation	Keeps research code consistent
Market example	Small price or portfolio example	Connects math to trading data
System-fit box	Where the idea lives in a research system	Turns a concept into implementation behavior
Confusions	Common mistakes and near-misses	Protects backtests from silent errors

STUDY RULE

Do not memorize returns as isolated formulas. Always ask: What input price field did I use? What timestamp alignment? Is the return simple or log? Is it forward-looking or backward-looking? Is it net of costs? Is it in decimals, percent, or basis points?

OUTCOME

By the end, raw prices should feel like the start of a pipeline, not the object of analysis. Quant research usually models returns, transformations of returns, labels derived from returns, or risk built from return distributions.

Table of Contents

NAVIGATION LOGIC

The manual progresses from units and ratios to return definitions, aggregation, portfolio math, data-quality traps, feature engineering, backtesting usage, diagnostics, and review.

PAGES

1	Title Page
2	How to Use This Manual
3	Table of Contents
4	Table of Contents - Continued
5	Notation and Formula Legend
6	Process Flow: Prices to Research Signals
7	Section Brief: Return Foundations
8	Arithmetic as Quant Infrastructure
9	Units, Scale, and Order of Operations
10	Ratios
11	Percent Change
12	Basis Points
13	Price Relatives and Gross Returns
14	Simple Returns
15	Log Returns
16	Arithmetic Return vs Log Return
17	Worked Calculation: Price to Return Series
18	Cumulative Simple Returns
19	Cumulative Log Returns
20	Compounding
21	Annualization
22	Arithmetic Mean, Geometric Mean, and Log Mean
23	Expected Return
24	Volatility and Return Dispersion
25	Return Standardization and Z-Scores
26	Excess Returns
27	Portfolio Weighted Returns
28	Rebalancing and Weight Drift
29	Leverage and Return Scaling
30	Transaction Costs in Return Space
31	Slippage, Spread, and Executable Prices
32	Adjusted Prices and Corporate Actions
33	Missing, Stale, and Bad Prices
34	Outliers and Winsorization
35	Rolling Returns
36	Lookback Windows and Horizon Discipline

Table of Contents - Continued

NAVIGATION LOGIC

The manual progresses from units and ratios to return definitions, aggregation, portfolio math, data-quality traps, feature engineering, backtesting usage, diagnostics, and review.

PAGES

37	Cross-Sectional Returns
38	Return Matrix
39	Correlation and Covariance from Returns
40	Beta from Returns
41	Alpha from Return Residuals
42	Forward Returns as Prediction Labels
43	Signal Decay and Return Horizons
44	Backtest Equity Curve
45	Drawdown from Returns
46	Regime Return Behavior
47	Return Distribution Diagnostics
48	Return Feature Engineering
49	Return Normalization Across Assets
50	Decimal Returns in Code
51	Return Alignment and Timestamp Ownership
52	Gross vs Net Returns
53	Return Smoothing and Hidden Risk
54	Approximation: Log Return About Equals Simple Return
55	Negative Returns and Lower Bounds
56	Cash Returns and Risk-Free Rate
57	Return Aggregation Rules
58	Sanity Checks for Return Columns
59	Decision Tree: Choosing a Return Transform
60	Feature Stability Map
61	Formula Summary Sheet
62	Failure-Mode Diagnostic Table
63	Implementation Checklist
64	Review Questions
65	Mini Index

Notation and Formula Legend

CORE NOTATION TABLE

Symbol	Meaning	System interpretation
P_t	Price at time t	A timestamped input from the market data store
P_{t-1}	Previous price	Requires correct sorting and calendar alignment
r_t	Simple return	Decimal change: 0.01 means +1 percent
G_t	Gross return	Growth factor: 1.01 means capital multiplied by 1.01
l_t	Log return	Natural log of the price relative
μ	Mean return	Center estimate; fragile at short samples
σ	Volatility / standard deviation	Dispersion of returns; core risk measure
w_i	Portfolio weight	Fraction of capital assigned to asset i
Σ	Covariance matrix	Risk interaction map across return series
α	Return not explained by benchmark beta	Often overfit if measured carelessly
β	Sensitivity to benchmark return	Market exposure estimate from returns

CORE RETURN IDENTITIES

```

r_t = P_t / P_{t-1} - 1
G_t = 1 + r_t = P_t / P_{t-1}
l_t = ln(G_t) = ln(P_t) - ln(P_{t-1})
cum simple = product(1 + r_t) - 1
cum log = sum(l_t)

```

These are the formulas that most of the manual builds on. The main implementation risk is mixing return types or mixing units.

UNIT DISCIPLINE

Store returns as decimals inside code. Convert to percent, basis points, or formatted strings only at the reporting layer. A return of 0.012 is 1.2 percent, or 120 bps. Confusing these units is one of the simplest ways to ruin sizing, volatility estimates, and reports.

RECALL CHECK

Which field is used: raw close, adjusted close, mid, bid, ask, or executable fill? Which timestamp owns the return: the ending bar, the starting bar, or the prediction horizon? Answer those before trusting any return column.

Process Flow: Prices to Research Signals

MEANING

Returns are the conversion layer between observed market prices and most research objects: features, labels, risk estimates, portfolio PnL, and validation metrics.

FULL-WIDTH PIPELINE DIAGRAM



PIPELINE FAILURE CHECKS

Failure point	Typical symptom	Fix
Wrong price field	Returns jump at split/dividend dates	Use adjusted prices or corporate-action logic
Lookahead label	Backtest looks too smooth	Shift forward returns correctly
Unit mistake	Leverage or risk sizing explodes	Store returns as decimals
Cost omission	Gross edge vanishes live	Subtract spread, slippage, fees, financing

SYSTEM FIT

In a meta research system, this page maps the boundary between raw data engineering and model research. Return transforms should be reusable, versioned, unit-tested, and logged as first-class pipeline components.

Section Brief: Return Foundations

SECTION ROLE

This section builds from arithmetic, ratios, and unit discipline into the specific return transforms used by quantitative research systems.

STUDY TARGETS

What you should know	Why it matters
Raw arithmetic	Needed before any return or PnL calculation can be trusted
Ratios and percent change	Normalize movement so assets can be compared
Simple and log returns	Define the basic language of performance and model targets
Compounding and aggregation	Explain how returns become equity curves and risk metrics

MEMORY RULE

A price series is not automatically a research series. You must decide the transform, the unit, the timestamp, the horizon, and the adjustment logic before it becomes trustworthy model input.

NEXT PAGES

The following pages define the basic arithmetic and return transforms used throughout feature engineering, portfolio construction, risk measurement, and backtest validation.

Arithmetic as Quant Infrastructure

MEANING AND MEMORY JOGGER

Meaning: Arithmetic is the basic calculation layer that turns prices, fills, capital, and weights into research variables.
Memory jogger: Before models, there is bookkeeping: add, subtract, multiply, divide, and keep units consistent.

PLAIN-ENGLISH EXPLANATION

Quant research looks advanced because the system around the math is complex. The first layer is still arithmetic. A return, a position size, a drawdown, a fee, and an equity curve are all arithmetic operations with strict timestamp and unit assumptions.

QUANT USE AND MARKET EXAMPLE

Quant use: Arithmetic is used in return construction, position sizing, PnL, cost adjustment, risk scaling, and summary statistics.
Simple market example: If price moves from 100 to 103, the arithmetic change is +3, but the return is $+3/100 = 0.03$.

FORMULA / CALCULATION

```
change = new_value - old_value
relative_change = change / old_value
new_value = old_value * (1 + return)
```

The denominator defines the interpretation. Dividing by the starting value gives a return; dividing by ending value gives a different object.

SYSTEM FIT AND IMPLEMENTATION

Research-system fit: Put arithmetic transforms in shared utility functions instead of rewriting them inside notebooks.
Python/system implementation idea: Use explicit names such as `dollar_change`, `pct_change_dec`, `gross_return`, and `log_return`. Add unit tests for known values.

CONFUSIONS, RELATED TERMS, RECALL

Confusions to avoid: Do not confuse dollar change with percent change. Do not mix display percent with decimal return.
Related terms: ratios, percent change, simple return, gross return, PnL, position sizing
Quick recall: What input, unit, horizon, and timestamp does this concept require?

VALIDATION NOTE

Validation note: write a small unit test with two or three prices before using the transform across a large research panel. Tiny examples catch most alignment and unit mistakes.

Units, Scale, and Order of Operations

MEANING AND MEMORY JOGGER

Meaning: Units define what a number means; scale defines its magnitude; order of operations defines how it is computed.
Memory jogger: A number without units is a bug waiting to happen.

PLAIN-ENGLISH EXPLANATION

The same value can mean 0.01 dollars, 1 percent, 1 basis point, or a decimal return. Research code often fails silently because the formula is correct but the unit is wrong. Parentheses matter because compounding and normalization are not interchangeable.

QUANT USE AND MARKET EXAMPLE

Quant use: Unit control protects features, risk estimates, leverage, and performance metrics from hidden scaling errors.
Simple market example: A model output of 0.75 could be a probability, z-score, ranking score, or 75 percent. The system must know which.

FORMULA / CALCULATION

```
percent = decimal_return * 100
basis_points = decimal_return * 10000
decimal_return = basis_points / 10000
```

Keep the internal unit canonical. Convert only at display or reporting boundaries.

SYSTEM FIT AND IMPLEMENTATION

Research-system fit: Define canonical units in the data schema and enforce them in validators.
Python/system implementation idea: Use metadata columns or naming conventions: `ret_dec`, `vol_ann_dec`, `fee_bps`, `price_usd`. Reject mixed-unit joins.

CONFUSIONS, RELATED TERMS, RECALL

Confusions to avoid: The expression $1 + 5$ percent is not 6; it is 1.05 as a growth factor. Basis points are not percent points.
Related terms: basis points, gross returns, annualized volatility, feature scaling
Quick recall: What input, unit, horizon, and timestamp does this concept require?

VALIDATION NOTE

Validation note: write a small unit test with two or three prices before using the transform across a large research panel. Tiny examples catch most alignment and unit mistakes.

Ratios

MEANING AND MEMORY JOGGER

Meaning: A ratio compares one quantity to another through division.

Memory jogger: A ratio answers: how many of this per one of that?

PLAIN-ENGLISH EXPLANATION

Ratios are everywhere in quant work because markets are relative. Price-to-price, profit-to-capital, risk-to-return, and exposure-to-equity are ratio ideas. The denominator controls the story. Changing the denominator changes the metric.

QUANT USE AND MARKET EXAMPLE

Quant use: Ratios power returns, Sharpe-like metrics, turnover, leverage, hit rate, drawdown, and factor exposures.

Simple market example: A strategy making 2 dollars on 100 dollars of capital has a $2/100 = 0.02$ return.

FORMULA / CALCULATION

```
ratio = numerator / denominator
return = price_change / starting_price
leverage = exposure / equity
```

A ratio is only meaningful when the numerator and denominator are aligned in time and definition.

SYSTEM FIT AND IMPLEMENTATION

Research-system fit: Use ratios to normalize raw quantities so they can be compared across assets and time.

Python/system implementation idea: Check for zero or near-zero denominators. Store denominator choice in metric documentation.

CONFUSIONS, RELATED TERMS, RECALL

Confusions to avoid: A ratio is not automatically causal or stable. A high ratio can be caused by a tiny denominator.

Related terms: percent change, return, leverage, normalization, Sharpe ratio

Quick recall: What input, unit, horizon, and timestamp does this concept require?

VALIDATION NOTE

Validation note: write a small unit test with two or three prices before using the transform across a large research panel. Tiny examples catch most alignment and unit mistakes.

Percent Change

MEANING AND MEMORY JOGGER

Meaning: Percent change measures relative change from an initial value.

Memory jogger: Change divided by where you started.

PLAIN-ENGLISH EXPLANATION

Percent change converts raw movement into a scale-free number. A 1 dollar move is large for a 5 dollar stock and tiny for a 500 dollar stock. Percent change fixes that by dividing by the starting value.

QUANT USE AND MARKET EXAMPLE

Quant use: It is the intuitive basis for simple returns, many momentum features, and performance reporting.

Simple market example: Price 50 to 55 is a +5 dollar move and a +10 percent change because $5/50 = 0.10$.

FORMULA / CALCULATION

```
percent_change = (new - old) / old
percent_display = percent_change * 100
```

In code, keep 0.10 as the return. Show 10 percent only in reports.

SYSTEM FIT AND IMPLEMENTATION

Research-system fit: Percent change is usually computed after price cleaning and before feature generation.

Python/system implementation idea: In pandas: `price.pct_change()` creates decimal simple returns. Shift carefully for forward labels.

CONFUSIONS, RELATED TERMS, RECALL

Confusions to avoid: A +50 percent move followed by -50 percent does not return to breakeven. Percent changes compound multiplicatively.

Related terms: simple return, gross return, compounding, drawdown

Quick recall: What input, unit, horizon, and timestamp does this concept require?

VALIDATION NOTE

Validation note: write a small unit test with two or three prices before using the transform across a large research panel. Tiny examples catch most alignment and unit mistakes.

Basis Points

MEANING AND MEMORY JOGGER

Meaning: A basis point is one one-hundredth of one percent, equal to 0.0001 in decimal return units.

Memory jogger: 1 bp = 0.01 percent = 0.0001 decimal.

PLAIN-ENGLISH EXPLANATION

Basis points make small rates, fees, spreads, and strategy edges easier to read. A daily edge of 0.0008 is easier to discuss as 8 bps. Execution costs are often naturally stated in bps.

QUANT USE AND MARKET EXAMPLE

Quant use: Used for transaction costs, spreads, borrow costs, yield changes, slippage, tracking error, and small average returns.

Simple market example: A 12 bp cost means subtract 0.0012 from return, not 0.12.

FORMULA / CALCULATION

```
bps = decimal_return * 10000
decimal_return = bps / 10000
25 bps = 0.25 percent = 0.0025
```

Basis points are a reporting unit. Do not multiply by 100 twice.

SYSTEM FIT AND IMPLEMENTATION

Research-system fit: Store costs in a cost model with explicit fields like `commission_bps` and `spread_bps`.

Python/system implementation idea: Write helper functions `bps_to_decimal(x)` and `decimal_to_bps(x)`. Use them at data boundaries.

CONFUSIONS, RELATED TERMS, RECALL

Confusions to avoid: Do not confuse 50 bps with 50 percent. It is 0.50 percent.

Related terms: percent, fees, spread, slippage, net returns

Quick recall: What input, unit, horizon, and timestamp does this concept require?

VALIDATION NOTE

Validation note: write a small unit test with two or three prices before using the transform across a large research panel. Tiny examples catch most alignment and unit mistakes.

Price Relatives and Gross Returns

MEANING AND MEMORY JOGGER

Meaning: A price relative or gross return is the ending price divided by the starting price.

Memory jogger: Gross return is the multiplier: 1.03 means capital became 103 percent of itself.

PLAIN-ENGLISH EXPLANATION

Gross returns are useful because compounding is multiplication. A simple return of +3 percent corresponds to a gross return of 1.03. A simple return of -3 percent corresponds to a gross return of 0.97.

QUANT USE AND MARKET EXAMPLE

Quant use: Gross returns are the cleanest object for equity curves, cumulative returns, and compounding.

Simple market example: Price 100 to 104 gives gross return $104/100 = 1.04$ and simple return 0.04.

FORMULA / CALCULATION

$$\begin{aligned} G_t &= P_t / P_{t-1} \\ r_t &= G_t - 1 \\ \text{equity}_t &= \text{equity}_{t-1} * G_t \end{aligned}$$

Gross return cannot be negative for long-only price relatives, but simple return can approach -1.

SYSTEM FIT AND IMPLEMENTATION

Research-system fit: A backtest equity curve is usually repeated multiplication of gross strategy returns.

Python/system implementation idea: Convert `net_ret` to `gross_net = 1 + net_ret` before compounding. Validate gross values are positive unless modeling unusual instruments.

CONFUSIONS, RELATED TERMS, RECALL

Confusions to avoid: Do not add gross returns. Multiply them over time.

Related terms: simple return, cumulative return, equity curve, compounding

Quick recall: What input, unit, horizon, and timestamp does this concept require?

VALIDATION NOTE

Validation note: write a small unit test with two or three prices before using the transform across a large research panel. Tiny examples catch most alignment and unit mistakes.

Simple Returns

MEANING AND MEMORY JOGGER

Meaning: A simple return is the percentage gain or loss from one price to the next.
Memory jogger: Simple return is percent change in decimal form.

PLAIN-ENGLISH EXPLANATION

Simple returns are intuitive and directly connect to capital. If you put 100 dollars into an asset and earn $r = 0.05$, you have 105 dollars before costs. They aggregate across assets through portfolio weights at the same timestamp.

QUANT USE AND MARKET EXAMPLE

Quant use: Used for PnL, portfolio returns, forward labels, risk metrics, and most performance reporting.
Simple market example: Buy at 20, sell at 22: $r = 22/20 - 1 = 0.10$, or +10 percent.

FORMULA / CALCULATION

```
r_t = P_t / P_{t-1} - 1
P_t = P_{t-1} * (1 + r_t)
portfolio_r_t = sum_i w_i * r_{i,t}
```

Simple returns add across assets at a point in time, but compound through time by multiplication.

SYSTEM FIT AND IMPLEMENTATION

Research-system fit: Use simple returns for realized capital growth, portfolio return aggregation, and net-of-cost backtest returns.
Python/system implementation idea: `price.pct_change()` returns simple returns. For forward labels use `price.shift(-h)/price - 1`.

CONFUSIONS, RELATED TERMS, RECALL

Confusions to avoid: Do not sum simple returns through time unless returns are tiny and approximation is acceptable.
Related terms: gross returns, log returns, portfolio weights, cumulative returns
Quick recall: What input, unit, horizon, and timestamp does this concept require?

VALIDATION NOTE

Validation note: write a small unit test with two or three prices before using the transform across a large research panel. Tiny examples catch most alignment and unit mistakes.

Log Returns

MEANING AND MEMORY JOGGER

Meaning: A log return is the natural log of the gross return.

Memory jogger: Log returns turn multiplication through time into addition.

PLAIN-ENGLISH EXPLANATION

Log returns are mathematically convenient because cumulative log return is the sum of period log returns. For small returns, log return and simple return are almost the same. For large moves, they diverge.

QUANT USE AND MARKET EXAMPLE

Quant use: Used in time-series modeling, distribution assumptions, additive compounding, and some volatility calculations.

Simple market example: Price 100 to 105: log return = $\ln(1.05)$ = about 0.04879, slightly less than simple return 0.05.

FORMULA / CALCULATION

$$l_t = \ln(P_t / P_{t-1})$$

$$l_t = \ln(1 + r_t)$$

$$\text{cumulative_log} = \sum_t l_t$$

To convert back to simple cumulative return: $\exp(\text{sum}(\text{log_returns})) - 1$.

SYSTEM FIT AND IMPLEMENTATION

Research-system fit: Use log returns when additive time aggregation helps, but convert back to simple returns for capital/PnL reporting.

Python/system implementation idea: `np.log(price).diff()` or `np.log1p(simple_ret)`. Guard against prices ≤ 0 .

CONFUSIONS, RELATED TERMS, RECALL

Confusions to avoid: Log returns do not aggregate across assets with portfolio weights the same way simple returns do.

Related terms: simple returns, gross returns, compounding, volatility

Quick recall: What input, unit, horizon, and timestamp does this concept require?

VALIDATION NOTE

Validation note: write a small unit test with two or three prices before using the transform across a large research panel. Tiny examples catch most alignment and unit mistakes.

Arithmetic Return vs Log Return

MEANING AND MEMORY JOGGER

Meaning: Simple return measures capital percentage change; log return measures logarithmic growth.

Memory jogger: Simple returns are capital-friendly; log returns are time-additive.

PLAIN-ENGLISH EXPLANATION

The two are close for small moves and different for large moves. A +50 percent simple return is $\ln(1.5) = 40.55$ percent log return. A -50 percent simple return is $\ln(0.5) = -69.31$ percent log return.

QUANT USE AND MARKET EXAMPLE

Quant use: Choosing the wrong return type can distort compounding, portfolio aggregation, risk estimates, and labels.

Simple market example: Two periods of +10 percent and -10 percent give simple cumulative return $1.1 \times 0.9 - 1 = -1$ percent; log returns sum to $\ln(0.99)$.

CORE COMPARISON

Question	Simple return	Log return
Aggregates over assets?	Yes, via weights	Not directly
Aggregates over time?	By product of $1+r$	By sum
Capital interpretation	Direct	Convert back with exp
Small-return approximation	r	about r

FORMULA / CALCULATION

simple: $r_t = P_t / P_{\{t-1\}} - 1$

log: $l_t = \ln(P_t / P_{\{t-1\}})$

convert: $r_t = \exp(l_t) - 1$

Simple and log returns are transformations of the same price relative, not separate market facts.

SYSTEM FIT AND IMPLEMENTATION

Research-system fit: Store both only when needed, with explicit names. Do not overwrite simple return columns with log returns.

Python/system implementation idea: Implement `return_type` as a parameter and write tests showing the difference for large moves.

CONFUSIONS, RELATED TERMS, RECALL

Confusions to avoid: Simple returns add across assets; log returns add through time. Do not swap those rules.

Related terms: price relatives, compounding, portfolio return, annualization

Quick recall: What input, unit, horizon, and timestamp does this concept require?

Worked Calculation: Price to Return Series

MEANING AND MEMORY JOGGER

Meaning: A return series is created by comparing each price to the prior aligned price.

Memory jogger: Sort, shift, divide, subtract one.

PLAIN-ENGLISH EXPLANATION

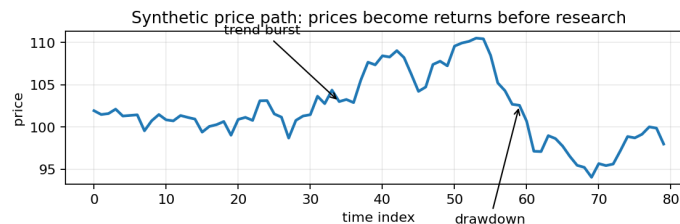
The mechanics are simple but the data discipline matters. Prices must be sorted by timestamp, adjusted consistently, grouped by asset, and aligned to the correct calendar before returns are trusted.

QUANT USE AND MARKET EXAMPLE

Quant use: This is the first transform that feeds momentum features, volatility estimates, labels, risk models, and backtests.

Simple market example: Prices [100, 102, 101] produce returns [missing, 0.0200, -0.009804].

SYNTHETIC PRICE CHART



Raw prices become research objects only after cleaning and return transformation.

FORMULA / CALCULATION

$P = [100, 102, 101]$

$r_2 = 102/100 - 1 = 0.0200$

$r_3 = 101/102 - 1 = -0.009804$

$\log_r_3 = \ln(101/102) = -0.009852$

The first return is missing because there is no previous price in the local series.

SYSTEM FIT AND IMPLEMENTATION

Research-system fit: The return builder should run after corporate action adjustment and before feature construction.

Python/system implementation idea: Group by symbol, sort by timestamp, compute pct_change, then validate missing first rows and extreme values.

CONFUSIONS, RELATED TERMS, RECALL

Confusions to avoid: Do not compute returns across different symbols, unsorted rows, or duplicated timestamps.

Related terms: pct_change, shift, adjusted prices, missing data

Quick recall: What input, unit, horizon, and timestamp does this concept require?

Cumulative Simple Returns

MEANING AND MEMORY JOGGER

Meaning: Cumulative simple return is total capital growth over multiple periods.

Memory jogger: Multiply the gross returns, then subtract one.

PLAIN-ENGLISH EXPLANATION

Simple returns do not add over time. A sequence of returns becomes a cumulative return by converting each to $1+r$, multiplying the gross returns, and subtracting one at the end.

QUANT USE AND MARKET EXAMPLE

Quant use: Used for equity curves, strategy performance, buy-and-hold benchmarks, and realized PnL reporting.

Simple market example: Returns +10 percent then -10 percent: $(1.10)(0.90)-1 = -0.01$, or -1 percent.

FORMULA / CALCULATION

```
cum_return = product_t(1 + r_t) - 1
equity_t = initial_capital * product_t(1 + r_t)
drawdown_t = equity_t / running_max(equity_t) - 1
```

The order of returns does not change final cumulative return if no cash flows occur, but it changes drawdown path.

SYSTEM FIT AND IMPLEMENTATION

Research-system fit: Backtest engines usually maintain an equity curve through cumulative simple returns.

Python/system implementation idea: Use $(1 + \text{ret}).\text{cumprod}() - 1$. Use net returns if costs are already deducted.

CONFUSIONS, RELATED TERMS, RECALL

Confusions to avoid: Adding +10 percent and -10 percent to get 0 percent is wrong for compounded capital.

Related terms: gross return, compounding, equity curve, drawdown

Quick recall: What input, unit, horizon, and timestamp does this concept require?

VALIDATION NOTE

Validation note: write a small unit test with two or three prices before using the transform across a large research panel. Tiny examples catch most alignment and unit mistakes.

Cumulative Log Returns

MEANING AND MEMORY JOGGER

Meaning: Cumulative log return is the sum of period log returns.

Memory jogger: Log returns compress the multiplication into addition.

PLAIN-ENGLISH EXPLANATION

Because log returns are logs of gross returns, summing them gives the log of the total gross return. Convert back with $\exp(\text{sum_log}) - 1$ when you need capital interpretation.

QUANT USE AND MARKET EXAMPLE

Quant use: Useful for time aggregation, continuous compounding intuition, and some statistical modeling workflows.

Simple market example: If log returns are 0.02 and -0.01, cumulative log return is 0.01 and simple cumulative return is $\exp(0.01) - 1$ = about 1.005 percent.

FORMULA / CALCULATION

```
cum_log = sum_t l_t
cum_simple = exp(cum_log) - 1
l_t = ln(1 + r_t)
```

Summed log return is not the same display object as cumulative simple return. Convert before reporting capital growth.

SYSTEM FIT AND IMPLEMENTATION

Research-system fit: Log-return pipelines should have explicit conversion to simple returns before equity/PnL visualization.

Python/system implementation idea: Use `np.log1p(simple_ret).cumsum()` and `np.exp(cum_log) - 1` for stable numerical conversion.

CONFUSIONS, RELATED TERMS, RECALL

Confusions to avoid: Do not report summed log returns as if they are simple returns without saying so.

Related terms: log returns, compounding, exp, equity curve

Quick recall: What input, unit, horizon, and timestamp does this concept require?

VALIDATION NOTE

Validation note: write a small unit test with two or three prices before using the transform across a large research panel. Tiny examples catch most alignment and unit mistakes.

Compounding

MEANING AND MEMORY JOGGER

Meaning: Compounding means returns apply to the updated value, not only the original value.

Memory jogger: Gains and losses change the base for the next period.

PLAIN-ENGLISH EXPLANATION

Compounding is why repeated small returns can create large differences over time, and why losses require larger gains to recover. It is also why backtests must use a path-based equity curve.

QUANT USE AND MARKET EXAMPLE

Quant use: Central to cumulative return, CAGR, drawdown, reinvestment, leverage, and portfolio growth.

Simple market example: Five periods of +1 percent creates $1.01^5 - 1 = 5.10$ percent, not exactly 5 percent.

FORMULA / CALCULATION

```
ending_capital = starting_capital * product_t(1 + r_t)
```

```
constant_r_cum = (1 + r)^n - 1
```

```
breakeven_gain_after_loss = loss / (1 - loss)
```

A 50 percent loss requires a 100 percent gain to recover because the base changed.

SYSTEM FIT AND IMPLEMENTATION

Research-system fit: Compounding logic belongs in backtest accounting, not just final reporting.

Python/system implementation idea: Track equity sequentially. Do not multiply returns by n unless using a deliberate approximation.

CONFUSIONS, RELATED TERMS, RECALL

Confusions to avoid: Average period return is not the same as compounded long-run growth.

Related terms: cumulative return, CAGR, drawdown, geometric mean

Quick recall: What input, unit, horizon, and timestamp does this concept require?

VALIDATION NOTE

Validation note: write a small unit test with two or three prices before using the transform across a large research panel. Tiny examples catch most alignment and unit mistakes.

Annualization

MEANING AND MEMORY JOGGER

Meaning: Annualization converts a period return or volatility estimate into an annual scale.

Memory jogger: Mean scales by time; volatility scales by square root of time under assumptions.

PLAIN-ENGLISH EXPLANATION

Annualization helps compare strategies across frequencies, but it assumes something about repeatability and independence. Annualizing a tiny, noisy sample can create fake precision.

QUANT USE AND MARKET EXAMPLE

Quant use: Used for annual return, annual volatility, Sharpe ratio, risk budgets, and strategy comparison.

Simple market example: Daily mean return of 0.0004 over 252 trading days gives approximate annual mean 10.08 percent.

FORMULA / CALCULATION

```
annual_mean_simple_approx = daily_mean * 252
```

```
annual_vol = daily_vol * sqrt(252)
```

```
CAGR = (ending/starting)^(252/n_days) - 1
```

Use CAGR for realized compounded growth. Use mean*252 for an arithmetic expectation approximation.

SYSTEM FIT AND IMPLEMENTATION

Research-system fit: Metric calculators should store frequency and trading-day assumptions explicitly.

Python/system implementation idea: Pass periods_per_year as a parameter: 252 for daily equities, 12 for monthly, custom for crypto or intraday.

CONFUSIONS, RELATED TERMS, RECALL

Confusions to avoid: Do not annualize a one-week backtest and treat it like a robust annual estimate.

Related terms: CAGR, volatility, Sharpe ratio, sampling frequency

Quick recall: What input, unit, horizon, and timestamp does this concept require?

VALIDATION NOTE

Validation note: write a small unit test with two or three prices before using the transform across a large research panel. Tiny examples catch most alignment and unit mistakes.

Arithmetic Mean, Geometric Mean, and Log Mean

MEANING AND MEMORY JOGGER

Meaning: Different averages answer different return questions.

Memory jogger: Arithmetic asks average period; geometric asks compounded growth; log mean asks average log growth.

PLAIN-ENGLISH EXPLANATION

The arithmetic mean is the simple average of returns. The geometric mean is the constant per-period return that gives the same total growth. The average log return is the mean of log growth rates.

QUANT USE AND MARKET EXAMPLE

Quant use: Mean choice affects strategy comparison, expected return estimates, optimization inputs, and long-run growth interpretation.

Simple market example: Returns +50 percent and -50 percent have arithmetic mean 0 percent but compounded return -25 percent.

FORMULA / CALCULATION

```
arith_mean = mean(r_t)
geom_mean = product(1+r_t)^(1/n) - 1
log_mean = mean(ln(1+r_t))
```

Volatility drag makes geometric mean lower than arithmetic mean when returns vary.

SYSTEM FIT AND IMPLEMENTATION

Research-system fit: Report arithmetic mean for period expectation and CAGR/geometric growth for realized path performance.

Python/system implementation idea: Use `np.nanmean` for arithmetic, `scipy.gmean` or `exp(mean(log1p(r)))-1` for geometric.

CONFUSIONS, RELATED TERMS, RECALL

Confusions to avoid: A positive arithmetic average does not guarantee positive compounded growth.

Related terms: volatility drag, CAGR, log returns, expected return

Quick recall: What input, unit, horizon, and timestamp does this concept require?

VALIDATION NOTE

Validation note: write a small unit test with two or three prices before using the transform across a large research panel. Tiny examples catch most alignment and unit mistakes.

Expected Return

MEANING AND MEMORY JOGGER

Meaning: Expected return is the probability-weighted average return you anticipate from a strategy or asset.

Memory jogger: Expectation is a weighted average of possible outcomes.

PLAIN-ENGLISH EXPLANATION

Expected return is not a promise. It is a model estimate. In quant research it can come from historical averages, forecasts, factor models, scenario analysis, or a predictive signal mapped to future returns.

QUANT USE AND MARKET EXAMPLE

Quant use: Expected return is the numerator of many allocation and ranking decisions.

Simple market example: If a trade has 60 percent chance of +2 percent and 40 percent chance of -1 percent, expected return is $0.6 \cdot 0.02 + 0.4 \cdot (-0.01) = 0.008$.

FORMULA / CALCULATION

```
E[r] = sum_i p_i * r_i
sample_mean = (1/n) * sum_t r_t
edge = expected_return - expected_cost
```

Expected return must be compared with uncertainty, costs, and capacity.

SYSTEM FIT AND IMPLEMENTATION

Research-system fit: A model forecast is often an expected return estimate, even if stored as a score.

Python/system implementation idea: Separate raw signal, calibrated expected return, and final portfolio weight. Log calibration assumptions.

CONFUSIONS, RELATED TERMS, RECALL

Confusions to avoid: High expected return with huge uncertainty may be unusable. Historical mean may be mostly noise.

Related terms: probability, sample mean, alpha, forecast, signal calibration

Quick recall: What input, unit, horizon, and timestamp does this concept require?

VALIDATION NOTE

Validation note: write a small unit test with two or three prices before using the transform across a large research panel. Tiny examples catch most alignment and unit mistakes.

Volatility and Return Dispersion

MEANING AND MEMORY JOGGER

Meaning: Volatility measures how widely returns vary around their average.
Memory jogger: Volatility is the size of the wiggle, not the direction.

PLAIN-ENGLISH EXPLANATION

Volatility is usually measured as standard deviation of returns. It is a risk proxy, not a full risk model. It does not fully describe tails, drawdowns, liquidity, or crash behavior.

QUANT USE AND MARKET EXAMPLE

Quant use: Used for risk scaling, position sizing, Sharpe ratio, volatility targeting, covariance matrices, and regime detection.
Simple market example: A strategy with daily returns around ± 0.5 percent is lower volatility than one swinging ± 3 percent.

FORMULA / CALCULATION

```
sigma = std(r_t)
annual_sigma = daily_sigma * sqrt(252)
z_t = (r_t - mu) / sigma
```

Volatility annualization assumes returns are reasonably independent and similarly distributed.

SYSTEM FIT AND IMPLEMENTATION

Research-system fit: Volatility features should be windowed, lagged, and computed without using future returns.
Python/system implementation idea: Use rolling std on past returns only. Shift the estimate if it feeds a next-period trading decision.

CONFUSIONS, RELATED TERMS, RECALL

Confusions to avoid: Low volatility is not the same as low risk if tail events are hidden.
Related terms: standard deviation, risk, Sharpe ratio, z-score, regimes
Quick recall: What input, unit, horizon, and timestamp does this concept require?

VALIDATION NOTE

Validation note: write a small unit test with two or three prices before using the transform across a large research panel. Tiny examples catch most alignment and unit mistakes.

Return Standardization and Z-Scores

MEANING AND MEMORY JOGGER

Meaning: Standardization converts returns or features into units of their own historical dispersion.

Memory jogger: A z-score says how many sigmas away from normal this value is.

PLAIN-ENGLISH EXPLANATION

Raw returns are not always comparable across assets because different assets have different volatility. Standardization subtracts a mean and divides by a standard deviation, creating a scale-relative feature.

QUANT USE AND MARKET EXAMPLE

Quant use: Used for feature normalization, anomaly detection, cross-asset comparison, and model input preprocessing.

Simple market example: A 1 percent return in a low-volatility asset can be more unusual than a 3 percent return in a high-volatility asset.

FORMULA / CALCULATION

```
z_t = (r_t - rolling_mean) / rolling_std
vol_scaled_return = r_t / rolling_sigma
clip_z = max(min(z_t, cap), -cap)
```

Use only past data for rolling mean and rolling standard deviation.

SYSTEM FIT AND IMPLEMENTATION

Research-system fit: Standardization belongs in the feature pipeline and must be versioned with window length and lag behavior.

Python/system implementation idea: Compute rolling statistics, shift by one bar, divide current return by the lagged statistic, and cap extremes.

CONFUSIONS, RELATED TERMS, RECALL

Confusions to avoid: Standardized does not mean predictive. It only changes scale.

Related terms: volatility, normalization, rolling window, outliers

Quick recall: What input, unit, horizon, and timestamp does this concept require?

VALIDATION NOTE

Validation note: write a small unit test with two or three prices before using the transform across a large research panel. Tiny examples catch most alignment and unit mistakes.

Excess Returns

MEANING AND MEMORY JOGGER

Meaning: Excess return is return above a benchmark, risk-free rate, or required hurdle.

Memory jogger: Excess means: what did you earn beyond the alternative?

PLAIN-ENGLISH EXPLANATION

Raw return can look good because the market or cash rate was favorable. Excess return subtracts the appropriate baseline to isolate incremental performance.

QUANT USE AND MARKET EXAMPLE

Quant use: Used for alpha estimation, Sharpe ratio, benchmark comparison, factor residuals, and portfolio evaluation.

Simple market example: If strategy return is 8 percent and benchmark return is 5 percent, excess return is 3 percent.

FORMULA / CALCULATION

```
excess_r_t = strategy_r_t - benchmark_r_t
excess_over_cash_t = r_t - rf_t
information_ratio = mean(active_r) / std(active_r)
```

Benchmark frequency and timestamp must match the strategy return frequency.

SYSTEM FIT AND IMPLEMENTATION

Research-system fit: Benchmark joins should be tested carefully; a one-day shift can fake alpha.

Python/system implementation idea: Align returns by date, left join benchmark, fill only according to a documented calendar rule, then subtract.

CONFUSIONS, RELATED TERMS, RECALL

Confusions to avoid: Excess over cash is different from excess over a risky benchmark.

Related terms: alpha, beta, benchmark, risk-free rate, active return

Quick recall: What input, unit, horizon, and timestamp does this concept require?

VALIDATION NOTE

Validation note: write a small unit test with two or three prices before using the transform across a large research panel. Tiny examples catch most alignment and unit mistakes.

Portfolio Weighted Returns

MEANING AND MEMORY JOGGER

Meaning: A portfolio simple return is the weighted sum of asset simple returns for the same period.

Memory jogger: Across assets: simple returns add by weights.

PLAIN-ENGLISH EXPLANATION

If weights are fractions of capital at the start of the period, the portfolio return is the sum of each asset weight times its simple return. This is why simple returns are natural for portfolio accounting.

QUANT USE AND MARKET EXAMPLE

Quant use: Used in portfolio construction, backtests, factor portfolios, risk decomposition, and allocation tests.

Simple market example: 60 percent in asset A returning 2 percent and 40 percent in asset B returning -1 percent gives $0.6 \cdot 0.02 + 0.4 \cdot (-0.01) = 0.008$.

FORMULA / CALCULATION

```
r_p,t = sum_i w_i,t-1 * r_i,t
sum_i w_i = 1 for fully invested long-only
equity_t = equity_{t-1} * (1 + r_p,t)
```

Weights must be known before the return period if this is a realistic backtest.

SYSTEM FIT AND IMPLEMENTATION

Research-system fit: Portfolio return calculation is the bridge between signal weights and equity curve.

Python/system implementation idea: Matrix multiply weights shifted by one period against realized asset returns. Include cash weight if not fully invested.

CONFUSIONS, RELATED TERMS, RECALL

Confusions to avoid: Do not use ending weights to compute the return that created those weights.

Related terms: simple returns, rebalancing, cash, leverage, PnL

Quick recall: What input, unit, horizon, and timestamp does this concept require?

VALIDATION NOTE

Validation note: write a small unit test with two or three prices before using the transform across a large research panel. Tiny examples catch most alignment and unit mistakes.

Rebalancing and Weight Drift

MEANING AND MEMORY JOGGER

Meaning: Weights change after returns unless the portfolio is rebalanced.

Memory jogger: Winners become a bigger share; losers become a smaller share.

PLAIN-ENGLISH EXPLANATION

A portfolio that starts 50/50 will not stay 50/50 after assets move differently. Rebalancing resets weights, creates turnover, and introduces costs.

QUANT USE AND MARKET EXAMPLE

Quant use: Rebalancing affects realized returns, risk exposure, turnover, cost drag, and factor purity.

Simple market example: If asset A rises and asset B falls, A becomes more than 50 percent of the portfolio before rebalancing.

FORMULA / CALCULATION

```
post_weight_i = w_i * (1 + r_i) / (1 + r_p)
turnover = 0.5 * sum_i abs(target_w_i - current_w_i)
cost = turnover * cost_rate
```

Post-return weights are mechanically caused by asset returns. Target weights are strategy decisions.

SYSTEM FIT AND IMPLEMENTATION

Research-system fit: Backtest engines need a clear sequence: observe data, form target weights, trade, pay costs, earn next return.

Python/system implementation idea: Represent current weights, target weights, fills, and post-return drift as separate states.

CONFUSIONS, RELATED TERMS, RECALL

Confusions to avoid: Ignoring drift can understate risk or costs. Rebalancing every bar may be unrealistic.

Related terms: portfolio return, turnover, transaction costs, execution

Quick recall: What input, unit, horizon, and timestamp does this concept require?

VALIDATION NOTE

Validation note: write a small unit test with two or three prices before using the transform across a large research panel. Tiny examples catch most alignment and unit mistakes.

Leverage and Return Scaling

MEANING AND MEMORY JOGGER

Meaning: Leverage scales exposure relative to capital, magnifying both gains and losses.

Memory jogger: Leverage multiplies the return path and the risk path.

PLAIN-ENGLISH EXPLANATION

A 2x leveraged position roughly doubles simple return before financing and margin effects. But compounding, volatility drag, costs, and constraints can make leveraged paths behave nonlinearly over time.

QUANT USE AND MARKET EXAMPLE

Quant use: Used in volatility targeting, futures strategies, long/short portfolios, and capital efficiency tests.

Simple market example: A 1 percent asset return with 3x exposure produces about 3 percent portfolio return before costs.

FORMULA / CALCULATION

```
levered_r_t = leverage * asset_r_t - financing_cost_t
gross_exposure = sum_i abs(position_value_i) / equity
net_exposure = sum_i position_value_i / equity
```

Leverage scaling is simple for one period but dangerous across volatile paths.

SYSTEM FIT AND IMPLEMENTATION

Research-system fit: Leverage should be applied through position sizing and risk controls, not by multiplying final backtest returns after the fact.

Python/system implementation idea: Calculate exposure each bar, enforce max leverage, margin, and borrow constraints before returns are realized.

CONFUSIONS, RELATED TERMS, RECALL

Confusions to avoid: Leverage does not create edge; it scales an existing return distribution.

Related terms: position sizing, risk, volatility targeting, drawdown

Quick recall: What input, unit, horizon, and timestamp does this concept require?

VALIDATION NOTE

Validation note: write a small unit test with two or three prices before using the transform across a large research panel. Tiny examples catch most alignment and unit mistakes.

Transaction Costs in Return Space

MEANING AND MEMORY JOGGER

Meaning: Transaction costs reduce realized returns and must be deducted from gross strategy returns.
Memory jogger: Edge is gross return minus the cost of getting it.

PLAIN-ENGLISH EXPLANATION

A strategy can look profitable before costs and fail after spreads, commissions, slippage, borrow, market impact, and financing. Costs are often small per trade but large relative to expected edge.

QUANT USE AND MARKET EXAMPLE

Quant use: Essential for backtests, turnover analysis, capacity estimates, and live performance realism.
Simple market example: If expected trade return is 6 bps and total cost is 8 bps, the net expected return is -2 bps.

FORMULA / CALCULATION

```
net_r_t = gross_r_t - cost_t
cost_t = turnover_t * cost_rate
edge_after_cost = expected_r - expected_cost
```

Costs should be timestamped when trades occur, not smeared conveniently after results.

SYSTEM FIT AND IMPLEMENTATION

Research-system fit: Cost models are part of the backtest engine and should be configurable by asset, venue, and period.
Python/system implementation idea: Subtract cost in decimal return units. Keep cost_bps, turnover, and net_ret columns separately.

CONFUSIONS, RELATED TERMS, RECALL

Confusions to avoid: Do not report gross Sharpe as strategy performance if the strategy requires trading.
Related terms: slippage, spread, turnover, net returns, capacity
Quick recall: What input, unit, horizon, and timestamp does this concept require?

VALIDATION NOTE

Validation note: write a small unit test with two or three prices before using the transform across a large research panel. Tiny examples catch most alignment and unit mistakes.

Slippage, Spread, and Executable Prices

MEANING AND MEMORY JOGGER

Meaning: Slippage is the difference between assumed execution price and realized or realistic execution price.

Memory jogger: The chart price is not always the trade price.

PLAIN-ENGLISH EXPLANATION

Mid, close, bid, ask, and fill prices are different objects. A strategy that buys at the close in a backtest may not get the close in reality unless the execution assumption is valid.

QUANT USE AND MARKET EXAMPLE

Quant use: Affects realized returns, short-horizon strategies, high-turnover systems, and capacity.

Simple market example: If the signal assumes buy at 100 but realistic fill is 100.05, the trade starts with a 5 bp disadvantage.

FORMULA / CALCULATION

```
slippage_return = (model_price - fill_price) / model_price for buys
effective_cost_bps = cost_decimal * 10000
net_fill_return = exit_fill / entry_fill - 1
```

Direction matters: a worse buy price and worse sell price both reduce return.

SYSTEM FIT AND IMPLEMENTATION

Research-system fit: Execution simulation should consume executable prices, not just research close prices.

Python/system implementation idea: Use bid/ask or conservative close-to-next-open assumptions. Separate alpha research from execution modeling.

CONFUSIONS, RELATED TERMS, RECALL

Confusions to avoid: Tiny spreads can dominate high-frequency or low-edge strategies.

Related terms: transaction costs, bid-ask spread, fills, market impact

Quick recall: What input, unit, horizon, and timestamp does this concept require?

VALIDATION NOTE

Validation note: write a small unit test with two or three prices before using the transform across a large research panel. Tiny examples catch most alignment and unit mistakes.

Adjusted Prices and Corporate Actions

MEANING AND MEMORY JOGGER

Meaning: Adjusted prices modify historical prices for splits, dividends, and similar events to create a consistent return series.
Memory jogger: A split is not a crash; a dividend is not always alpha.

PLAIN-ENGLISH EXPLANATION

Raw prices can jump because of corporate actions rather than economic gains or losses. Adjusted prices try to represent the investor return path more consistently.

QUANT USE AND MARKET EXAMPLE

Quant use: Critical for equity return data, long backtests, factor research, and dividend-paying assets.
Simple market example: A 2-for-1 split can make raw price fall from 100 to 50 while the investor did not lose 50 percent.

FORMULA / CALCULATION

```
raw_return may include artificial jumps
adjusted_return = adjusted_close_t / adjusted_close_{t-1} - 1
total_return includes price change plus distributions
```

Adjustment methodology differs by data vendor; document it.

SYSTEM FIT AND IMPLEMENTATION

Research-system fit: Corporate action handling belongs in the data ingestion and cleaning layer before return construction.
Python/system implementation idea: Prefer vendor-adjusted close for daily equity research, but verify split and dividend behavior on known dates.

CONFUSIONS, RELATED TERMS, RECALL

Confusions to avoid: Adjusted close is not always executable price. It is a research series.
Related terms: simple returns, total return, dividends, splits, data cleaning
Quick recall: What input, unit, horizon, and timestamp does this concept require?

VALIDATION NOTE

Validation note: write a small unit test with two or three prices before using the transform across a large research panel. Tiny examples catch most alignment and unit mistakes.

Missing, Stale, and Bad Prices

MEANING AND MEMORY JOGGER

Meaning: Return calculations fail when price observations are missing, duplicated, stale, or wrong.
Memory jogger: Bad prices create fake returns.

PLAIN-ENGLISH EXPLANATION

A missing price can create gaps. A stale price can create zero returns followed by a fake jump. A bad print can create an extreme return that contaminates features and risk estimates.

QUANT USE AND MARKET EXAMPLE

Quant use: Data quality directly affects volatility, covariance, labels, backtest trades, and model training.
Simple market example: If yesterday price is stale at 100 and today updates to 110 after several missing bars, the computed return may assign a multi-day move to one bar.

FORMULA / CALCULATION

```
return requires valid P_t and P_{t-1}
gap_return_h = P_t / P_{t-h} - 1
zero_return_streak may indicate stale data
```

A computed return is only as trustworthy as both prices and the calendar gap between them.

SYSTEM FIT AND IMPLEMENTATION

Research-system fit: Create data-quality flags before return generation and propagate them into features.
Python/system implementation idea: Check duplicate timestamps, missing bars, zero-volume bars, extreme jumps, and stale repeats.

CONFUSIONS, RELATED TERMS, RECALL

Confusions to avoid: Forward filling prices can create artificial zero returns and delayed jumps.
Related terms: data cleaning, calendar alignment, outliers, adjusted prices
Quick recall: What input, unit, horizon, and timestamp does this concept require?

VALIDATION NOTE

Validation note: write a small unit test with two or three prices before using the transform across a large research panel. Tiny examples catch most alignment and unit mistakes.

Outliers and Winsorization

MEANING AND MEMORY JOGGER

Meaning: An outlier is an unusually extreme value; winsorization caps extremes at chosen thresholds.

Memory jogger: One bad tick can dominate a model.

PLAIN-ENGLISH EXPLANATION

Some extreme returns are real events and some are data errors. Blindly deleting extremes can remove the thing you need to model. Blindly keeping bad prints can destroy estimates.

QUANT USE AND MARKET EXAMPLE

Quant use: Outlier handling affects volatility, correlations, regressions, feature scaling, and risk controls.

Simple market example: A daily stock return of +900 percent may be a real microcap event, a split adjustment error, or a bad data record.

FORMULA / CALCULATION

```
winsorized_r = min(max(r, lower_cap), upper_cap)
z = (r - rolling_mean) / rolling_std
flag if abs(z) > threshold
```

Capping is a modeling decision, not a data-cleaning substitute.

SYSTEM FIT AND IMPLEMENTATION

Research-system fit: Use separate columns for raw_return, cleaned_return, and outlier_flag to preserve auditability.

Python/system implementation idea: Flag extremes by asset and regime. Review samples before applying global caps.

CONFUSIONS, RELATED TERMS, RECALL

Confusions to avoid: Do not confuse rare but real tail risk with bad data.

Related terms: data validation, volatility, tail risk, robust statistics

Quick recall: What input, unit, horizon, and timestamp does this concept require?

VALIDATION NOTE

Validation note: write a small unit test with two or three prices before using the transform across a large research panel. Tiny examples catch most alignment and unit mistakes.

Rolling Returns

MEANING AND MEMORY JOGGER

Meaning: A rolling return measures performance over a moving lookback window.
Memory jogger: At each date, ask: how much did it move over the last h periods?

PLAIN-ENGLISH EXPLANATION

Rolling returns convert a price series into overlapping window measurements. They are common in momentum, trend, reversal, and performance monitoring.

QUANT USE AND MARKET EXAMPLE

Quant use: Used for momentum features, trailing performance, trend filters, and research diagnostics.
Simple market example: A 20-day rolling return is $P_t / P_{t-20} - 1$.

FORMULA / CALCULATION

```
rolling_ret_h,t = P_t / P_{t-h} - 1
rolling_log_h,t = ln(P_t) - ln(P_{t-h})
feature_t should use only data available at t
```

Overlapping rolling returns are serially correlated. Treat sample size carefully.

SYSTEM FIT AND IMPLEMENTATION

Research-system fit: Rolling features belong in the feature store with explicit horizon and lag.
Python/system implementation idea: Use `price / price.shift(h) - 1`. Shift the feature if decisions happen after feature observation.

CONFUSIONS, RELATED TERMS, RECALL

Confusions to avoid: A rolling return is not the same as summing daily simple returns over the window.
Related terms: lookback window, momentum, log returns, feature engineering
Quick recall: What input, unit, horizon, and timestamp does this concept require?

VALIDATION NOTE

Validation note: write a small unit test with two or three prices before using the transform across a large research panel. Tiny examples catch most alignment and unit mistakes.

Lookback Windows and Horizon Discipline

MEANING AND MEMORY JOGGER

Meaning: A lookback window defines how much past data a feature uses; a horizon defines what future period a label measures.
Memory jogger: Lookback is past; horizon is future.

PLAIN-ENGLISH EXPLANATION

Confusing lookback and horizon causes lookahead bias or meaningless features. A 20-day momentum feature and a 5-day forward return label answer different questions.

QUANT USE AND MARKET EXAMPLE

Quant use: Core to predictive modeling, walk-forward tests, signal decay analysis, and validation.
Simple market example: Feature: last 20-day return at date t . Label: next 5-day return from t to $t+5$.

FORMULA / CALCULATION

```
feature_t = P_t / P_{t-20} - 1
label_t = P_{t+5} / P_t - 1
do not let feature_t use P_{t+1...t+5}
```

The timestamp t is the decision boundary. Everything after t is future information.

SYSTEM FIT AND IMPLEMENTATION

Research-system fit: Research datasets should encode lookback and forecast horizon in feature and label names.
Python/system implementation idea: Use `shift(-h)` for forward labels and `shift(1)` when features require next-bar execution.

CONFUSIONS, RELATED TERMS, RECALL

Confusions to avoid: A model can look excellent if the feature accidentally contains the label window.
Related terms: forward returns, labels, lookahead bias, walk-forward validation
Quick recall: What input, unit, horizon, and timestamp does this concept require?

VALIDATION NOTE

Validation note: write a small unit test with two or three prices before using the transform across a large research panel. Tiny examples catch most alignment and unit mistakes.

Cross-Sectional Returns

MEANING AND MEMORY JOGGER

Meaning: Cross-sectional returns compare many assets over the same time period.

Memory jogger: Same timestamp, many assets.

PLAIN-ENGLISH EXPLANATION

Instead of studying one asset through time, cross-sectional work asks which assets outperformed others over the same interval. This is common in ranking, factor models, and long/short portfolios.

QUANT USE AND MARKET EXAMPLE

Quant use: Used for factor research, relative strength, market-neutral strategies, and ranking models.

Simple market example: On the same day, stock A returns +2 percent, B returns -1 percent, and C returns +0.5 percent. A ranks highest.

FORMULA / CALCULATION

```
cross_section_t = {r_{1,t}, r_{2,t}, ..., r_{N,t}}
rank_i,t = rank(r_{i,t} among assets at t)
demeaned_r_i,t = r_i,t - mean_cross_section_t
```

Cross-sectional transforms require matching timestamps and a defined universe.

SYSTEM FIT AND IMPLEMENTATION

Research-system fit: Universe construction, survivorship control, and timestamp alignment are prerequisites.

Python/system implementation idea: Group by date, rank or z-score across symbols, and avoid using assets unavailable at that date.

CONFUSIONS, RELATED TERMS, RECALL

Confusions to avoid: Do not mix cross-sectional ranks from different dates or universes.

Related terms: ranking, factor returns, universe, market neutral

Quick recall: What input, unit, horizon, and timestamp does this concept require?

VALIDATION NOTE

Validation note: write a small unit test with two or three prices before using the transform across a large research panel. Tiny examples catch most alignment and unit mistakes.

Return Matrix

MEANING AND MEMORY JOGGER

Meaning: A return matrix organizes returns as time by asset so models can operate on panels.

Memory jogger: Rows are dates; columns are assets.

PLAIN-ENGLISH EXPLANATION

Many quant operations need a rectangular matrix: covariance, correlations, PCA, factor models, portfolio optimization, and cross-sectional signals. Missing values and calendar alignment become visible in this format.

QUANT USE AND MARKET EXAMPLE

Quant use: Return matrices are the core data structure for multi-asset research.

Simple market example: A 252 x 500 matrix might contain one year of daily returns for 500 stocks.

FORMULA / CALCULATION

R shape = T x N

row t = returns across assets at time t

column i = return history for asset i

Matrix shape and missing-value policy affect every downstream estimate.

SYSTEM FIT AND IMPLEMENTATION

Research-system fit: A feature store or research engine should produce validated return matrices by universe and frequency.

Python/system implementation idea: Pivot long data to wide with index=date, columns=symbol, values=return. Track missingness by row and column.

CONFUSIONS, RELATED TERMS, RECALL

Confusions to avoid: Filling missing values with zero can imply no price movement when data may be absent.

Related terms: correlation, covariance, cross-sectional returns, panel data

Quick recall: What input, unit, horizon, and timestamp does this concept require?

VALIDATION NOTE

Validation note: write a small unit test with two or three prices before using the transform across a large research panel. Tiny examples catch most alignment and unit mistakes.

Correlation and Covariance from Returns

MEANING AND MEMORY JOGGER

Meaning: Covariance measures joint movement; correlation standardizes covariance into a -1 to +1 scale.

Memory jogger: Covariance has units; correlation is normalized co-movement.

PLAIN-ENGLISH EXPLANATION

These measures are calculated from return series, not raw prices. Correlation tells whether assets tend to move together. Covariance includes the magnitude of movement and is used directly in portfolio risk.

QUANT USE AND MARKET EXAMPLE

Quant use: Core for risk models, diversification, hedging, factor exposure, clustering, and portfolio optimization.

Simple market example: Two assets with high positive correlation often lose diversification benefit when held together.

FORMULA / CALCULATION

```
cov(i,j) = E[(r_i - mu_i)(r_j - mu_j)]
corr(i,j) = cov(i,j) / (sigma_i * sigma_j)
portfolio_var = w.T * Sigma * w
```

Correlation is unstable when estimated from short samples or during regime changes.

SYSTEM FIT AND IMPLEMENTATION

Research-system fit: Risk engines should estimate covariance using cleaned, aligned, and lag-appropriate returns.

Python/system implementation idea: Use rolling or exponentially weighted covariance. Monitor missing data and sample size.

CONFUSIONS, RELATED TERMS, RECALL

Confusions to avoid: Correlation is not static. Crisis correlations often rise.

Related terms: return matrix, volatility, covariance matrix, portfolio risk

Quick recall: What input, unit, horizon, and timestamp does this concept require?

VALIDATION NOTE

Validation note: write a small unit test with two or three prices before using the transform across a large research panel. Tiny examples catch most alignment and unit mistakes.

Beta from Returns

MEANING AND MEMORY JOGGER

Meaning: Beta estimates how sensitive an asset or strategy is to a benchmark return.

Memory jogger: Beta asks: how much does this move when the benchmark moves?

PLAIN-ENGLISH EXPLANATION

Beta is usually estimated by regressing asset returns on benchmark returns. A beta near 1 means similar directional exposure. A beta near 0 means little linear benchmark exposure, not necessarily no risk.

QUANT USE AND MARKET EXAMPLE

Quant use: Used for hedging, alpha estimation, factor exposure, risk decomposition, and performance attribution.

Simple market example: If a stock often moves 1.2 percent when the market moves 1 percent, its market beta may be around 1.2.

FORMULA / CALCULATION

```
r_i,t = alpha + beta * r_m,t + error_t  
beta = cov(r_i, r_m) / var(r_m)  
hedged_r = r_i - beta * r_m
```

Beta depends on frequency, lookback window, benchmark, and regime.

SYSTEM FIT AND IMPLEMENTATION

Research-system fit: Exposure reports should include beta estimates with sample dates and benchmark identifiers.

Python/system implementation idea: Run rolling regressions on aligned returns. Lag or freeze beta estimates before using them for trades.

CONFUSIONS, RELATED TERMS, RECALL

Confusions to avoid: Low beta does not imply positive alpha. It only describes benchmark sensitivity.

Related terms: alpha, covariance, regression, benchmark, hedging

Quick recall: What input, unit, horizon, and timestamp does this concept require?

VALIDATION NOTE

Validation note: write a small unit test with two or three prices before using the transform across a large research panel. Tiny examples catch most alignment and unit mistakes.

Alpha from Return Residuals

MEANING AND MEMORY JOGGER

Meaning: Alpha is return not explained by chosen risk exposures or benchmark movement.

Memory jogger: Alpha is what remains after the benchmark explanation.

PLAIN-ENGLISH EXPLANATION

Alpha is often estimated as an intercept or residual return after controlling for beta or factors. It is easy to overstate because omitted risks, costs, data snooping, and regime luck can masquerade as alpha.

QUANT USE AND MARKET EXAMPLE

Quant use: Central to strategy evaluation, factor research, manager analysis, and model validation.

Simple market example: If a strategy earns 10 percent while its benchmark exposure explains 7 percent, the unexplained 3 percent may be candidate alpha.

FORMULA / CALCULATION

```
r_strategy = alpha + beta * r_benchmark + error
active_return = r_strategy - r_benchmark
net_alpha = gross_alpha - costs - financing
```

Alpha depends on the benchmark model. Change the benchmark and the alpha estimate changes.

SYSTEM FIT AND IMPLEMENTATION

Research-system fit: Store alpha estimates together with benchmark, factor set, cost model, sample window, and statistical confidence.

Python/system implementation idea: Use out-of-sample periods and factor regressions. Do not tune the benchmark to flatter the strategy.

CONFUSIONS, RELATED TERMS, RECALL

Confusions to avoid: Backtested alpha is not proof of real alpha. It can be overfit noise.

Related terms: beta, excess return, residual, factor model, backtesting

Quick recall: What input, unit, horizon, and timestamp does this concept require?

VALIDATION NOTE

Validation note: write a small unit test with two or three prices before using the transform across a large research panel. Tiny examples catch most alignment and unit mistakes.

Forward Returns as Prediction Labels

MEANING AND MEMORY JOGGER

Meaning: A forward return is the return from the current time into a future horizon.
Memory jogger: Forward return is the thing the model is trying to predict.

PLAIN-ENGLISH EXPLANATION

Predictive models need labels. A forward return label must be calculated from future prices relative to the decision time, then carefully aligned so the model never sees it as an input.

QUANT USE AND MARKET EXAMPLE

Quant use: Used for supervised learning, ranking targets, signal evaluation, and information coefficient tests.
Simple market example: A 5-day forward return at date t is $P_{t+5}/P_t - 1$.

FORMULA / CALCULATION

```
fwd_ret_h,t = P_{t+h} / P_t - 1
fwd_log_h,t = ln(P_{t+h}) - ln(P_t)
label_t must not enter feature_t
```

The label uses future data, but only as the target during training and validation.

SYSTEM FIT AND IMPLEMENTATION

Research-system fit: Dataset builders should enforce feature/label separation with column namespaces and timestamp checks.
Python/system implementation idea: Use `shift(-h)` within each symbol. Drop rows where future price is unavailable. Confirm by spot-checking dates.

CONFUSIONS, RELATED TERMS, RECALL

Confusions to avoid: A one-row alignment mistake can turn the target into a feature and fake performance.
Related terms: lookahead bias, labels, horizon, supervised learning
Quick recall: What input, unit, horizon, and timestamp does this concept require?

VALIDATION NOTE

Validation note: write a small unit test with two or three prices before using the transform across a large research panel. Tiny examples catch most alignment and unit mistakes.

Signal Decay and Return Horizons

MEANING AND MEMORY JOGGER

Meaning: Signal decay measures how predictive power changes as the forward return horizon changes.

Memory jogger: A signal has a half-life; test the clock it lives on.

PLAIN-ENGLISH EXPLANATION

A feature may predict the next day, next week, or next month differently. Measuring return horizons helps identify whether a signal is fast, slow, unstable, or just noise.

QUANT USE AND MARKET EXAMPLE

Quant use: Used for horizon selection, turnover estimates, holding period design, and strategy timing.

Simple market example: A short-term reversal signal may work at 1-day horizon but disappear by 20 days.

FORMULA / CALCULATION

```
IC_h = corr(signal_t, fwd_ret_h,t)
decay curve = IC by horizon h
holding_period should match signal persistence
```

Longer horizons reduce sample independence because forward returns overlap.

SYSTEM FIT AND IMPLEMENTATION

Research-system fit: Model evaluation should produce horizon panels, not just one chosen label.

Python/system implementation idea: Compute labels for multiple h values, evaluate IC or spread return by h, and inspect decay stability by regime.

CONFUSIONS, RELATED TERMS, RECALL

Confusions to avoid: Selecting the best horizon after seeing results is another form of overfitting.

Related terms: forward returns, information coefficient, turnover, holding period

Quick recall: What input, unit, horizon, and timestamp does this concept require?

VALIDATION NOTE

Validation note: write a small unit test with two or three prices before using the transform across a large research panel. Tiny examples catch most alignment and unit mistakes.

Backtest Equity Curve

MEANING AND MEMORY JOGGER

Meaning: An equity curve tracks strategy capital through time after applying returns.
Memory jogger: Returns are the steps; equity is the path.

PLAIN-ENGLISH EXPLANATION

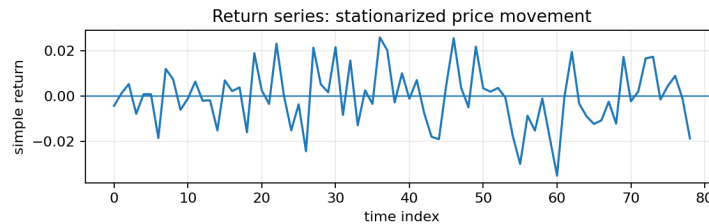
The equity curve is produced by compounding strategy returns through time. It should reflect net returns after costs, financing, and realistic execution assumptions.

QUANT USE AND MARKET EXAMPLE

Quant use: Used for performance reporting, drawdown analysis, risk monitoring, and strategy comparison.

Simple market example: Starting with 100, returns +2 percent, -1 percent, +3 percent create equity 100 -> 102 -> 100.98 -> 104.0094.

RETURN SERIES CHART



The equity curve is just compounded net returns; return quality controls the whole backtest.

FORMULA / CALCULATION

```
equity_t = equity_{t-1} * (1 + strategy_ret_t)
cum_return_t = equity_t / equity_0 - 1
net_strategy_ret = gross_ret - cost_ret
```

Equity should be path-based, not reconstructed from average return alone.

SYSTEM FIT AND IMPLEMENTATION

Research-system fit: The backtest engine should produce returns, costs, positions, trades, and equity as separate audit tables.

Python/system implementation idea: Use a sequential accounting loop or vectorized cumprod after all net returns are computed correctly.

CONFUSIONS, RELATED TERMS, RECALL

Confusions to avoid: A smooth equity curve can still be fragile if trade count is low or assumptions are unrealistic.

Related terms: cumulative return, transaction costs, drawdown, validation

Quick recall: What input, unit, horizon, and timestamp does this concept require?

Drawdown from Returns

MEANING AND MEMORY JOGGER

Meaning: Drawdown measures decline from a prior equity peak.

Memory jogger: Drawdown asks: how far below the high-water mark are you?

PLAIN-ENGLISH EXPLANATION

Drawdown is path-dependent. Two strategies can have the same final return but different drawdowns depending on the order of gains and losses. This makes it essential for risk assessment.

QUANT USE AND MARKET EXAMPLE

Quant use: Used for risk limits, strategy comparison, capital allocation, investor reporting, and stop logic.

Simple market example: If equity peaks at 120 and falls to 90, drawdown is $90/120 - 1 = -25$ percent.

FORMULA / CALCULATION

```
running_peak_t = max(equity_0...equity_t)
drawdown_t = equity_t / running_peak_t - 1
max_drawdown = min_t drawdown_t
```

Drawdown is computed from compounded equity, not directly from individual returns.

SYSTEM FIT AND IMPLEMENTATION

Research-system fit: Risk dashboards should calculate current drawdown, max drawdown, drawdown duration, and recovery time.

Python/system implementation idea: `equity = (1+ret).cumprod()`; `dd = equity/equity.cummax()-1`.

CONFUSIONS, RELATED TERMS, RECALL

Confusions to avoid: Volatility and drawdown are related but not identical. Drawdown depends on sequence.

Related terms: equity curve, compounding, risk, recovery

Quick recall: What input, unit, horizon, and timestamp does this concept require?

VALIDATION NOTE

Validation note: write a small unit test with two or three prices before using the transform across a large research panel. Tiny examples catch most alignment and unit mistakes.

Regime Return Behavior

MEANING AND MEMORY JOGGER

Meaning: A regime is a market state where return behavior changes meaningfully.

Memory jogger: Same formula, different environment.

PLAIN-ENGLISH EXPLANATION

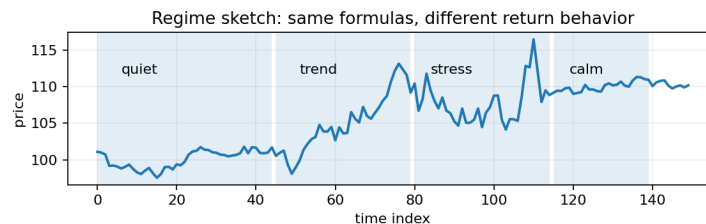
Returns can behave differently in trending, mean-reverting, quiet, volatile, risk-on, or risk-off periods. A signal that works in one regime may fail in another.

QUANT USE AND MARKET EXAMPLE

Quant use: Used for robustness testing, conditional models, risk limits, and strategy monitoring.

Simple market example: Momentum may perform well in persistent trends and poorly in choppy mean-reverting periods.

REGIME DIAGRAM



Visual regime labels help you test whether a return pattern is stable or environment-dependent.

FORMULA / CALCULATION

```
regime_t = function(volatility_t, trend_t, correlation_t, liquidity_t)
conditional_mean = mean(r_t | regime)
conditional_risk = std(r_t | regime)
```

Regime labels are approximations. Avoid overfitting the regime definition.

SYSTEM FIT AND IMPLEMENTATION

Research-system fit: Validation should slice returns by regime to reveal hidden dependency.

Python/system implementation idea: Compute regime features from past data, assign labels, and report performance by label.

CONFUSIONS, RELATED TERMS, RECALL

Confusions to avoid: A regime filter built from future information creates lookahead bias.

Related terms: volatility, rolling windows, robustness, walk-forward testing

Quick recall: What input, unit, horizon, and timestamp does this concept require?

Return Distribution Diagnostics

MEANING AND MEMORY JOGGER

Meaning: A return distribution shows how often different return outcomes occur.
Memory jogger: Do not only ask average; ask shape.

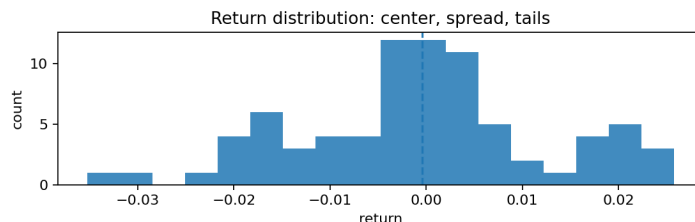
PLAIN-ENGLISH EXPLANATION

Returns may have skew, fat tails, outliers, volatility clustering, and asymmetry. A mean and standard deviation can hide important risks.

QUANT USE AND MARKET EXAMPLE

Quant use: Used for model assumptions, risk controls, stress tests, tail analysis, and strategy comparison.
Simple market example: A strategy with many tiny wins and rare huge losses may have a pleasant average until the tail arrives.

RETURN DISTRIBUTION CHART



Distribution shape reveals risks that average return hides.

FORMULA / CALCULATION

```
skew = asymmetry of returns
kurtosis = tail heaviness proxy
tail_loss_q = quantile(return, q)
```

Distribution estimates are sample-dependent and unstable in the tails.

SYSTEM FIT AND IMPLEMENTATION

Research-system fit: Diagnostics should include histogram, quantiles, tail events, worst periods, and regime slices.
Python/system implementation idea: Compute quantiles, skew, kurtosis, hit rate, average win/loss, and worst n returns. Review extreme rows.

CONFUSIONS, RELATED TERMS, RECALL

Confusions to avoid: A normal-looking histogram does not prove returns are normal.
Related terms: volatility, outliers, tail risk, VaR, drawdown
Quick recall: What input, unit, horizon, and timestamp does this concept require?

Return Feature Engineering

MEANING AND MEMORY JOGGER

Meaning: Return feature engineering transforms historical returns into model inputs.
Memory jogger: Features are structured questions about the past.

PLAIN-ENGLISH EXPLANATION

A return can become momentum, reversal, volatility, skew, rank, z-score, drawdown, or regime state. Each feature encodes a hypothesis about future returns.

QUANT USE AND MARKET EXAMPLE

Quant use: Used to build predictive models, rule-based strategies, factor portfolios, and diagnostics.
Simple market example: A 20-day momentum feature uses the last 20-day return to ask whether recent winners continue winning.

FORMULA / CALCULATION

```
momentum_20 = P_t / P_{t-20} - 1
reversal_5 = -1 * (P_t / P_{t-5} - 1)
vol_20 = std(r_{t-19}...r_t)
```

Feature definitions must be known before validation. Avoid tuning hundreds of variations without controls.

SYSTEM FIT AND IMPLEMENTATION

Research-system fit: Feature definitions should be versioned and reproducible, with input columns and lookback windows recorded.
Python/system implementation idea: Create feature factories with parameterized windows. Save metadata: window, lag, return type, universe, cleaning policy.

CONFUSIONS, RELATED TERMS, RECALL

Confusions to avoid: More features do not automatically mean more signal; they often mean more overfitting opportunity.
Related terms: momentum, reversal, volatility, labels, feature store
Quick recall: What input, unit, horizon, and timestamp does this concept require?

VALIDATION NOTE

Validation note: write a small unit test with two or three prices before using the transform across a large research panel. Tiny examples catch most alignment and unit mistakes.

Return Normalization Across Assets

MEANING AND MEMORY JOGGER

Meaning: Normalization makes return-derived features more comparable across assets.
Memory jogger: Compare unusualness, not just raw size.

PLAIN-ENGLISH EXPLANATION

Different assets have different typical movement sizes. A raw 2 percent return means different things for a utility stock, crypto asset, or leveraged ETF.

QUANT USE AND MARKET EXAMPLE

Quant use: Used in cross-sectional ranking, ML preprocessing, risk parity, and anomaly detection.
Simple market example: A 1 percent move with 0.5 percent daily vol is a 2-sigma event; with 4 percent daily vol it is ordinary.

FORMULA / CALCULATION

```
vol_norm_ret_t = r_t / rolling_sigma_t  
rank_norm_t = rank(feature_t across universe)  
demeaned_t = feature_t - group_mean_t
```

Normalization choice changes what the model learns.

SYSTEM FIT AND IMPLEMENTATION

Research-system fit: Normalize using past-only statistics and keep raw features available for audit.
Python/system implementation idea: Compute by asset, by group, or cross-sectionally depending on the research question.

CONFUSIONS, RELATED TERMS, RECALL

Confusions to avoid: Normalization can remove useful level information if applied blindly.
Related terms: z-score, volatility, cross-sectional returns, feature scaling
Quick recall: What input, unit, horizon, and timestamp does this concept require?

VALIDATION NOTE

Validation note: write a small unit test with two or three prices before using the transform across a large research panel. Tiny examples catch most alignment and unit mistakes.

Decimal Returns in Code

MEANING AND MEMORY JOGGER

Meaning: Decimal return representation stores 1 percent as 0.01.

Memory jogger: Code uses decimals; humans read percent.

PLAIN-ENGLISH EXPLANATION

Most formulas assume decimal returns. Display formatting should not leak back into calculations. Confusing 0.01 with 1 or 100 creates severe errors.

QUANT USE AND MARKET EXAMPLE

Quant use: Essential for performance metrics, risk models, cost calculations, and reporting.

Simple market example: A 25 bp fee is 0.0025 decimal, not 25 and not 0.25.

FORMULA / CALCULATION

1 percent = 0.01

100 bps = 0.01

reported_percent = decimal_return * 100

Unit conversion belongs at boundaries: data input and report output.

SYSTEM FIT AND IMPLEMENTATION

Research-system fit: A schema should tag unit: decimal_return, percent_display, bps_cost, dollars.

Python/system implementation idea: Use conversion helpers and assert ranges. A daily return above 1.0 should usually be flagged.

CONFUSIONS, RELATED TERMS, RECALL

Confusions to avoid: Pandas pct_change returns decimal returns, not percent values.

Related terms: percent change, basis points, data schema, validation

Quick recall: What input, unit, horizon, and timestamp does this concept require?

VALIDATION NOTE

Validation note: write a small unit test with two or three prices before using the transform across a large research panel. Tiny examples catch most alignment and unit mistakes.

Return Alignment and Timestamp Ownership

MEANING AND MEMORY JOGGER

Meaning: Timestamp ownership defines which time index a return belongs to.
Memory jogger: The return from $t-1$ to t is usually stamped at t .

PLAIN-ENGLISH EXPLANATION

A return compares two prices. The timestamp convention matters because features, labels, and trades must align. A return stamped at the wrong time can leak future information.

QUANT USE AND MARKET EXAMPLE

Quant use: Critical for supervised learning, backtesting, event studies, and intraday strategies.
Simple market example: Close-to-close return from Monday close to Tuesday close is usually stored on Tuesday.

FORMULA / CALCULATION

```
ret_t = P_t / P_{t-1} - 1
feature_t uses information available at or before t
trade_{t+1} should not use return_{t+1}
```

A timestamp is not only a label; it defines information availability.

SYSTEM FIT AND IMPLEMENTATION

Research-system fit: Centralize timestamp conventions in the research data contract.
Python/system implementation idea: Document whether bars are labeled by open time, close time, or event time. Write alignment tests.

CONFUSIONS, RELATED TERMS, RECALL

Confusions to avoid: A feature can be mathematically correct and still impossible to trade if timestamped incorrectly.
Related terms: lookahead bias, labels, bars, backtesting
Quick recall: What input, unit, horizon, and timestamp does this concept require?

VALIDATION NOTE

Validation note: write a small unit test with two or three prices before using the transform across a large research panel. Tiny examples catch most alignment and unit mistakes.

Gross vs Net Returns

MEANING AND MEMORY JOGGER

Meaning: Gross return is before costs; net return is after costs and frictions.

Memory jogger: Gross is the idea; net is what capital gets.

PLAIN-ENGLISH EXPLANATION

Gross returns help inspect raw signal potential, but net returns are what matter for actual strategy viability. Cost assumptions should be explicit and stress-tested.

QUANT USE AND MARKET EXAMPLE

Quant use: Used for separating research alpha from implementation drag.

Simple market example: A signal spread of 15 bps with 6 bps estimated cost leaves 9 bps net before other risks.

FORMULA / CALCULATION

```
net_return = gross_return - trading_cost - financing_cost
net_gross_factor = 1 + net_return
net_equity = product(1 + net_return)
```

Net returns should include all costs relevant to the strategy horizon and instrument.

SYSTEM FIT AND IMPLEMENTATION

Research-system fit: Performance reports should show gross, costs, and net side by side.

Python/system implementation idea: Compute cost components separately, then aggregate to `net_ret`. Preserve each component for diagnostics.

CONFUSIONS, RELATED TERMS, RECALL

Confusions to avoid: A strategy is not profitable because gross returns are positive.

Related terms: transaction costs, slippage, financing, capacity

Quick recall: What input, unit, horizon, and timestamp does this concept require?

VALIDATION NOTE

Validation note: write a small unit test with two or three prices before using the transform across a large research panel. Tiny examples catch most alignment and unit mistakes.

Return Smoothing and Hidden Risk

MEANING AND MEMORY JOGGER

Meaning: Return smoothing happens when marks or assumptions reduce visible volatility without reducing real risk.
Memory jogger: Smooth returns can be fake comfort.

PLAIN-ENGLISH EXPLANATION

Illiquid assets, stale prices, optimistic marks, or accounting assumptions can make returns look safer than they are. Smoothing suppresses volatility and drawdown until a repricing occurs.

QUANT USE AND MARKET EXAMPLE

Quant use: Important for private assets, thinly traded securities, backtests using stale quotes, and risk monitoring.
Simple market example: A stale price series may show several zero returns then one large jump when the price updates.

FORMULA / CALCULATION

`observed_r_t` may understate `true_r_t`
`zero_return_runs` can signal stale marks
`unsmoothed risk` > `observed risk`

Measured volatility can be artificially low when prices do not update.

SYSTEM FIT AND IMPLEMENTATION

Research-system fit: Data-quality monitors should flag stale prices, zero-return runs, and suspiciously low volatility.
Python/system implementation idea: Track last update timestamp, volume, quote age, and zero-return streaks by asset.

CONFUSIONS, RELATED TERMS, RECALL

Confusions to avoid: Low observed volatility is not proof of low economic risk.
Related terms: stale prices, volatility, liquidity, drawdown
Quick recall: What input, unit, horizon, and timestamp does this concept require?

VALIDATION NOTE

Validation note: write a small unit test with two or three prices before using the transform across a large research panel. Tiny examples catch most alignment and unit mistakes.

Approximation: Log Return About Equals Simple Return

MEANING AND MEMORY JOGGER

Meaning: For small returns, log return is approximately equal to simple return.
Memory jogger: Tiny moves make log and simple nearly the same.

PLAIN-ENGLISH EXPLANATION

The approximation comes from $\ln(1+r)$ being close to r when r is small. This is why daily liquid-asset returns often look similar in either form.

QUANT USE AND MARKET EXAMPLE

Quant use: Useful for mental math and some modeling assumptions, but dangerous for large moves or leveraged returns.
Simple market example: If $r = 0.001$, log return $\ln(1.001)$ is about 0.0009995.

FORMULA / CALCULATION

$\ln(1 + r)$ approx r for small r
 $\ln(1 + r) = r - r^2/2 + r^3/3 - \dots$
 difference grows as $\text{abs}(r)$ grows

The second-order term explains why log return is lower than simple return for positive r .

SYSTEM FIT AND IMPLEMENTATION

Research-system fit: Approximation rules should not replace exact transformations in production code.
Python/system implementation idea: Use `np.log1p(r)` for exact log conversion instead of manual approximation.

CONFUSIONS, RELATED TERMS, RECALL

Confusions to avoid: Approximation is not identity. Large crypto, option, or leveraged moves can diverge materially.
Related terms: log return, simple return, Taylor approximation
Quick recall: What input, unit, horizon, and timestamp does this concept require?

VALIDATION NOTE

Validation note: write a small unit test with two or three prices before using the transform across a large research panel. Tiny examples catch most alignment and unit mistakes.

Negative Returns and Lower Bounds

MEANING AND MEMORY JOGGER

Meaning: Simple returns for long-only assets are bounded below by -100 percent.
Memory jogger: You can lose all capital, not more, without leverage.

PLAIN-ENGLISH EXPLANATION

A simple return of -1 means the asset went to zero. Log return approaches negative infinity as price approaches zero. Leveraged or short positions can lose more than initial capital depending on structure.

QUANT USE AND MARKET EXAMPLE

Quant use: Important for risk, model targets, transformations, and sanity checks.
Simple market example: Price 10 to 0 gives simple return -1. Log return is undefined at zero.

FORMULA / CALCULATION

```
r_t = P_t/P_{t-1} - 1 >= -1 when P_t >= 0
log_return undefined if P_t <= 0
levered_return can be less than -1
```

Log returns require strictly positive prices.

SYSTEM FIT AND IMPLEMENTATION

Research-system fit: Return validation should flag prices ≤ 0 and returns below -1 unless leverage or derivative mechanics explain them.
Python/system implementation idea: Before log transforms, assert price > 0 . For leveraged portfolios, validate equity and margin states.

CONFUSIONS, RELATED TERMS, RECALL

Confusions to avoid: A return less than -100 percent may be possible for a leveraged portfolio but not for an unlevered long-only price relative.
Related terms: log return, leverage, validation, price cleaning
Quick recall: What input, unit, horizon, and timestamp does this concept require?

VALIDATION NOTE

Validation note: write a small unit test with two or three prices before using the transform across a large research panel. Tiny examples catch most alignment and unit mistakes.

Cash Returns and Risk-Free Rate

MEANING AND MEMORY JOGGER

Meaning: Cash return or risk-free return is the baseline return from holding cash or a low-risk instrument.

Memory jogger: Cash is not always zero.

PLAIN-ENGLISH EXPLANATION

When rates are material, ignoring cash return can distort excess return, Sharpe ratio, and portfolio performance. Cash matters when strategies sit out of the market or use leverage.

QUANT USE AND MARKET EXAMPLE

Quant use: Used in excess returns, financing costs, Sharpe ratio, futures collateral, and cash allocation.

Simple market example: A strategy earning 4 percent when cash earns 5 percent has negative excess return over cash.

FORMULA / CALCULATION

```
excess_r_t = r_t - rf_t
portfolio_r = risky_weight*risky_r + cash_weight*cash_r
financing_cost = borrowed_notional * rate_period
```

Match the risk-free rate frequency to the return frequency.

SYSTEM FIT AND IMPLEMENTATION

Research-system fit: Backtests should include cash return and financing assumptions explicitly.

Python/system implementation idea: Join a daily or period risk-free series and convert annual yield into period return consistently.

CONFUSIONS, RELATED TERMS, RECALL

Confusions to avoid: Zero cash return may be acceptable in some simplified tests but not in all rate regimes.

Related terms: excess returns, Sharpe ratio, leverage, financing

Quick recall: What input, unit, horizon, and timestamp does this concept require?

VALIDATION NOTE

Validation note: write a small unit test with two or three prices before using the transform across a large research panel. Tiny examples catch most alignment and unit mistakes.

Return Aggregation Rules

MEANING AND MEMORY JOGGER

Meaning: Returns aggregate differently across time, assets, and scenarios.

Memory jogger: Time, assets, and scenarios have different math.

PLAIN-ENGLISH EXPLANATION

Simple returns multiply through time and weight across assets. Log returns add through time. Scenario expectations probability-weight outcomes. Mixing these rules causes misleading metrics.

QUANT USE AND MARKET EXAMPLE

Quant use: A core mental model for portfolio math, backtesting, and risk reporting.

Simple market example: Across assets use $0.6*r_A + 0.4*r_B$. Through time use $\text{product}(1+r_t)-1$.

FORMULA / CALCULATION

```
time simple: product(1+r_t)-1
asset simple: sum_i w_i*r_i,t
time log: sum_t log_r_t
scenario expectation: sum_i p_i*r_i
```

Always ask what dimension you are aggregating over.

SYSTEM FIT AND IMPLEMENTATION

Research-system fit: Metric code should name aggregation dimension: `time_agg`, `cross_section_agg`, `scenario_agg`.

Python/system implementation idea: Use separate functions for time compounding, portfolio weighting, and scenario expectation.

CONFUSIONS, RELATED TERMS, RECALL

Confusions to avoid: Do not average across assets and call it a time-series return without knowing weights.

Related terms: simple returns, log returns, portfolio weights, expectation

Quick recall: What input, unit, horizon, and timestamp does this concept require?

VALIDATION NOTE

Validation note: write a small unit test with two or three prices before using the transform across a large research panel. Tiny examples catch most alignment and unit mistakes.

Sanity Checks for Return Columns

MEANING AND MEMORY JOGGER

Meaning: Sanity checks detect impossible, suspicious, or mis-scaled return data.
Memory jogger: Trust returns only after they survive basic abuse tests.

PLAIN-ENGLISH EXPLANATION

Before modeling, a return column should be checked for missingness, extreme values, unit scale, date ordering, first-row behavior, stale periods, and known corporate action dates.

QUANT USE AND MARKET EXAMPLE

Quant use: Prevents garbage features, fake alpha, broken covariance, and misleading charts.
Simple market example: If daily returns are often greater than 10, the column may be percent values stored as decimals.

FORMULA / CALCULATION

```
max_abs_return threshold by asset type  
missing_rate = missing_count / total_count  
zero_streak_length as stale-price flag
```

Thresholds should be asset-class aware. Crypto and penny stocks differ from large-cap ETFs.

SYSTEM FIT AND IMPLEMENTATION

Research-system fit: Make sanity checks part of the pipeline, not optional notebook inspection.
Python/system implementation idea: Build automated validation reports with warnings and failing thresholds.

CONFUSIONS, RELATED TERMS, RECALL

Confusions to avoid: A clean-looking chart can hide wrong units or alignment.
Related terms: data validation, outliers, adjusted prices, unit discipline
Quick recall: What input, unit, horizon, and timestamp does this concept require?

VALIDATION NOTE

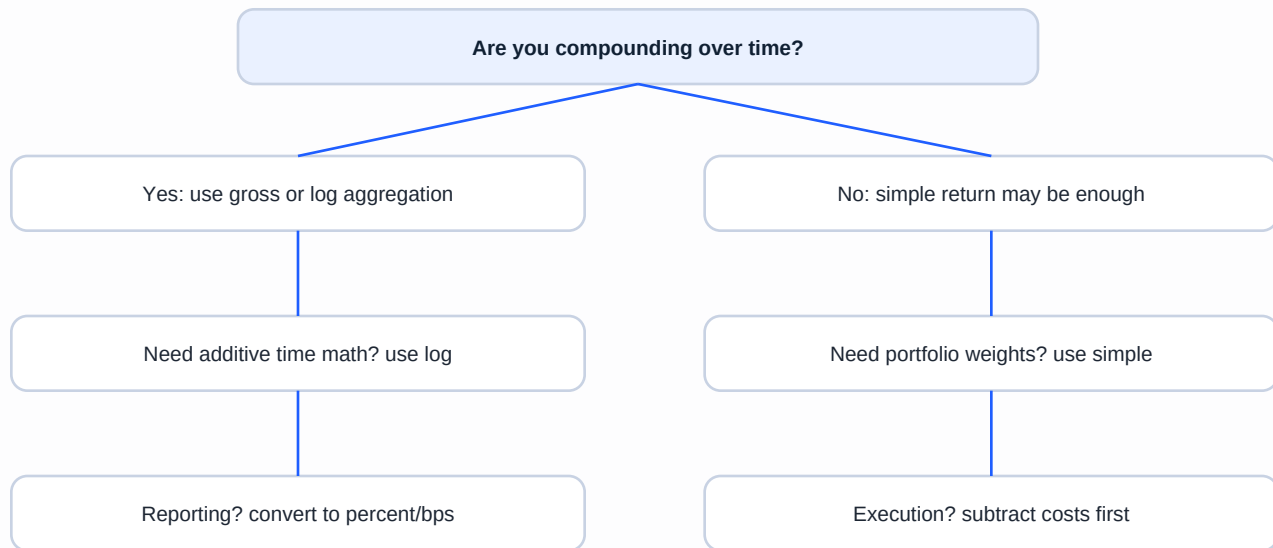
Validation note: write a small unit test with two or three prices before using the transform across a large research panel. Tiny examples catch most alignment and unit mistakes.

Decision Tree: Choosing a Return Transform

DECISION PURPOSE

This is not a law. It is a practical research decision aid for choosing simple returns, log returns, gross returns, or basis-point units without mixing concepts.

FULL-WIDTH DECISION TREE



Always decide units last: internal calculations should usually stay in decimal returns.

CHOICE SUMMARY

Need	Preferred form	Why
Single-period PnL	simple return	Matches weighted portfolio returns
Multi-period additive analysis	log return	Sums across time
Growth of capital	gross return	Multiplicative compounding
Human reporting	percent or bps	Readable units
Execution research	net simple return	Costs affect realized capital

IMPLEMENTATION RULE

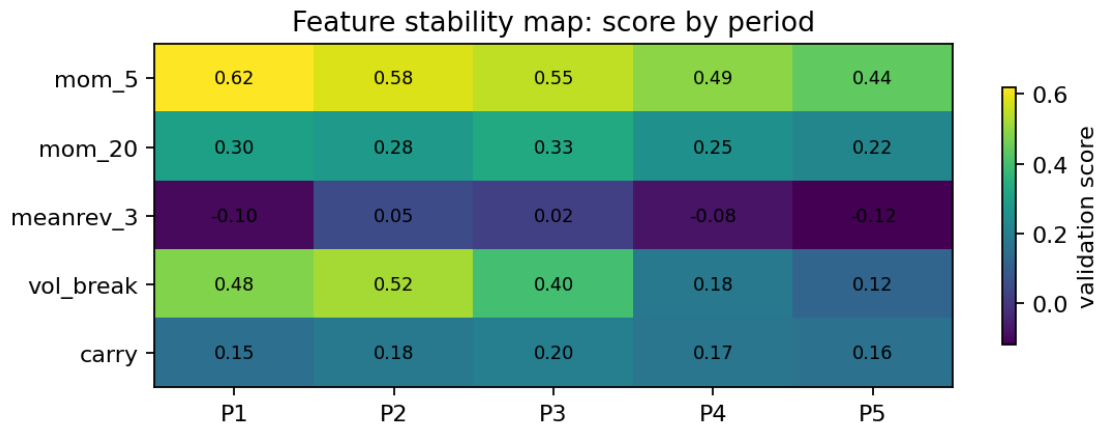
Keep separate column names: `ret_1d_simple`, `logret_1d`, `fwd_ret_5d`, `excess_ret_1d`, `net_ret_1d`. Generic names like `return` or `r` invite silent misuse.

Feature Stability Map

WHAT THIS VISUAL CHECKS

A return-derived feature should not only look good in one slice. Check whether its validation score survives across periods, regimes, assets, and parameter choices. Stability does not prove future profitability, but instability is a strong warning sign.

FEATURE STABILITY HEATMAP



Rows are features; columns are validation periods. Watch for features that only work in one isolated window.

INTERPRETATION GUIDE

Pattern	Interpretation	Action
Stable positive row	Candidate robust feature	Test costs, capacity, and out-of-sample behavior
One bright cell	Likely period-specific luck	Do not promote without explanation
Decaying row	Signal may be arbitrated or regime-specific	Test horizon and regime dependence
Sign flips	Feature is unstable or conditional	Slice by regime and universe

SYSTEM CONNECTION

Feature stability reports belong after backtest validation and before deployment. Store the score, sample, feature version, universe, and cost assumptions so you can compare future research honestly.

BOTTOM-LINE APPLICATION CHECK

Application check: take one return column from a real or synthetic dataset and state its price input, timestamp convention, horizon, unit, return type, cost treatment, and whether it is safe as a feature or only as a label. If any answer is vague, the column is not research-grade yet.

Formula Summary Sheet

RETURN FORMULAS

```
change = P_t - P_{t-1}
simple_r = P_t / P_{t-1} - 1
gross_G = 1 + simple_r
log_l = ln(P_t / P_{t-1})
simple_from_log = exp(log_l) - 1
```

Use simple returns for capital and portfolio weights; use log returns for additive time aggregation.

AGGREGATION FORMULAS

```
cum_simple = product(1 + r_t) - 1
cum_log = sum(log_l_t)
portfolio_r = sum_i w_i * r_i
equity_t = equity_{t-1} * (1 + net_r_t)
```

The aggregation dimension determines the formula: time, assets, scenarios, or reporting units.

FORMULA COMPARISON TABLE

Object	Formula	Use
Simple return	$P_t/P_{t-1}-1$	Capital change, portfolio weighting
Gross return	$1+r$	Compounding and equity curves
Log return	$\ln(1+r)$	Additive time aggregation
Basis points	$r*10000$	Costs, spreads, small edges
Excess return	$r - \text{benchmark}$	Alpha and active performance

RECALL RULE

Never ask only which formula is correct. Ask which object you need: capital growth, additive log growth, weighted portfolio return, benchmark-relative return, cost-adjusted return, or display unit.

BOTTOM-LINE APPLICATION CHECK

Application check: take one return column from a real or synthetic dataset and state its price input, timestamp convention, horizon, unit, return type, cost treatment, and whether it is safe as a feature or only as a label. If any answer is vague, the column is not research-grade yet.

Failure-Mode Diagnostic Table

WHY THIS PAGE EXISTS

Most beginner quant return errors are not advanced math errors. They are unit errors, timestamp errors, price-field errors, and cost omissions. This page is an audit checklist.

FAILURE-MODE DIAGNOSTIC TABLE

Symptom	Likely cause	Fix
Daily returns around 2 or 5 instead of 0.02 or 0.05	Percent values stored as decimals	Normalize units and enforce ranges
Huge return on split date	Raw close used instead of adjusted series	Use adjusted prices or action data
Excellent ML validation with suspiciously high accuracy	Forward return leaked into features	Rebuild alignment and shift tests
Backtest edge disappears live	Costs, spread, or slippage omitted	Model net executable returns
Low volatility with sudden jumps	Stale prices or forward-filled data	Flag stale bars and quote age
Portfolio return too high	Ending weights used or leverage double-counted	Use prior weights and exposure audit
Cumulative return inconsistent with equity	Added simple returns over time	Use cumprod of gross returns

DEBUGGING SEQUENCE

Start with three rows for one asset. Print price, previous price, simple return, log return, forward label, feature timestamp, and trade timestamp. If the tiny example is wrong, the full panel is wrong.

SYSTEM CONNECTION

A production research system should fail fast when return validation fails. Warnings are not enough for core transforms that feed labels, risk, and backtests.

BOTTOM-LINE APPLICATION CHECK

Application check: take one return column from a real or synthetic dataset and state its price input, timestamp convention, horizon, unit, return type, cost treatment, and whether it is safe as a feature or only as a label. If any answer is vague, the column is not research-grade yet.

Implementation Checklist

CORE IMPLEMENTATION CHECKLIST

Use this checklist before trusting any return-based research table. It is designed for a feature store, backtest engine, or notebook research pipeline.

PIPELINE CHECKLIST

Step	Question	Pass condition
Price input	Which price field is used?	Field is documented and appropriate for research/execution
Sorting	Are rows sorted within symbol?	No cross-symbol pct_change or duplicate timestamp bug
Adjustment	Are splits/dividends handled?	Known action dates do not create fake jumps
Return type	Simple, gross, log, excess, net?	Column name says exact object
Units	Decimal, percent, or bps?	Internal calculations use decimal returns
Alignment	Does timestamp represent ending period?	Features and labels respect information boundary
Costs	Are trading costs included where needed?	Gross and net returns reported separately
Validation	Are ranges and missingness checked?	Automated report passes before modeling

NAMING CONVENTION

Recommended names: ret_1d_simple, logret_1d, fwd_ret_5d_simple, excess_ret_1d, net_ret_1d, cost_bps, gross_ret, equity_curve, drawdown. Avoid vague names like return, signal_return, or perf.

UNIT TEST IDEA

Create a tiny table with prices [100, 102, 101]. Expected simple returns are [NaN, 0.020000, -0.009804]. Expected log returns are [NaN, $\ln(1.02)$, $\ln(101/102)$]. This one test catches many broken pipelines.

BOTTOM-LINE APPLICATION CHECK

Application check: take one return column from a real or synthetic dataset and state its price input, timestamp convention, horizon, unit, return type, cost treatment, and whether it is safe as a feature or only as a label. If any answer is vague, the column is not research-grade yet.

Review Questions

QUICK RECALL QUESTIONS

1. Why can simple returns be weighted across assets but not added through time?
2. Why can log returns be added through time but not directly used for weighted portfolio returns?
3. What does 25 bps mean in decimal return units?
4. Why is adjusted close useful for research but not automatically executable?
5. What does it mean for a return label to be forward-looking?
6. Why can high gross return vanish after costs?
7. What is the difference between arithmetic mean return and geometric mean return?
8. How can stale prices create fake low volatility?

MINI EXAMPLES

Problem	Answer
Price 80 to 84	simple return = $84/80 - 1 = 0.05$
Return 0.003 in bps	30 bps
Two returns +10 percent and -10 percent	cumulative = $1.1 \times 0.9 - 1 = -1$ percent
Log returns 0.01 and 0.02	cumulative simple = $\exp(0.03) - 1$
Portfolio 70 percent at +1 percent, 30 percent at -2 percent	return = $0.007 - 0.006 = 0.001$

COMMON CONFUSIONS TO DRILL

Decimal vs percent. Gross vs simple. Simple vs log. Past lookback vs future horizon. Gross vs net. Raw close vs adjusted close. Feature timestamp vs label timestamp. Average return vs compounded growth.

STUDY PROMPT

Pick one stock or ETF. Build a 10-row table by hand with price, simple return, log return, gross return, cumulative return, and drawdown. Then reproduce the same table in Python. If both match, you understand the mechanics.

BOTTOM-LINE APPLICATION CHECK

Application check: take one return column from a real or synthetic dataset and state its price input, timestamp convention, horizon, unit, return type, cost treatment, and whether it is safe as a feature or only as a label. If any answer is vague, the column is not research-grade yet.

Mini Index

MINI INDEX - RETURN OBJECTS

Arithmetic change: dollar movement. Ratio: one quantity divided by another. Percent change: relative movement from start value. Simple return: decimal percent change. Gross return: multiplier $1+r$. Log return: natural log of gross return. Cumulative return: total compounded growth. Excess return: return above benchmark or cash. Net return: return after costs.

MINI INDEX - SYSTEM OBJECTS

Price cleaner: fixes raw data before returns. Return builder: converts prices into return columns. Feature store: saves lagged return-derived signals. Label builder: creates forward returns for modeling. Backtest engine: turns weights and returns into PnL. Risk engine: computes volatility, covariance, beta, drawdown, and exposure.

RELATED CONCEPT MAP

Concept cluster	Connects to
Return transforms	percent change, simple, gross, log, cumulative
Portfolio math	weights, rebalancing, leverage, transaction costs
Data quality	adjusted prices, missing data, stale prices, outliers
Modeling	features, forward returns, labels, normalization
Risk	volatility, covariance, beta, drawdown, regimes

CLOSING RULE

In quant research, returns are not just formulas. They are data contracts. Each return column should say what price field it used, what horizon it spans, what unit it uses, whether it is simple or log, whether it is gross or net, and whether it is safe for features or labels.

BOTTOM-LINE APPLICATION CHECK

Application check: take one return column from a real or synthetic dataset and state its price input, timestamp convention, horizon, unit, return type, cost treatment, and whether it is safe as a feature or only as a label. If any answer is vague, the column is not research-grade yet.